WaterGuard: mapping flood damage from space

Adil Mahajan. Code: see README. Case study: Bhote Koshi–Trishuli glacier-outburst flood, Nepal, 25/26 Aug 2026 (Copernicus EMS EMSR927).

1. Problem and what we built

When roads, bridges and phone lines fail, rescuers need three answers fast: **where** the flood hit, **what** it damaged and **who** is cut off. WaterGuard answers them for any area and date, from free satellite data only:

- an **impact map** of flooded and debris-covered ground, fusing Sentinel-1 radar (cloud-proof), Sentinel-2 optical change (where clear) and the Copernicus DEM (where a flood can physically reach);
- a **damage overlay** of pre-event OpenStreetMap buildings, roads, bridges and health facilities;
- a **cut-off analysis** that removes impacted road segments and damaged bridges from the road graph and finds settlements that lost every link to a town, a health facility or the wider network;
- a **situation-report copilot** (English + Nepali) that answers only from these results, with an automatic number checker;
- an **inspector agent** that re-examines the map with OpenCV 5 in a perceive → decide → act loop (re-align, zoom, look again weeks later) and sends whatever it cannot confirm to a human;
- **verifiable reports**: SHA-256 Merkle fingerprints anchored on a public chain, which anyone can re-run and check;
- a **dashboard**, a printable **one-page situation report** (EN/NE) and a **flood-path tracer** (bonus).

2. Data rules

Input	Source	Role
Sentinel-1 RTC (VV, VH)	Microsoft Planetary Computer (Meru Labs fallback)	change detection, all orbit tracks
Sentinel-2 L2A	Planetary Computer	change detection where clear
Copernicus DEM GLO-30	Planetary Computer	HAND, slope, layover/shadow, flow paths
OpenStreetMap, snapshot 24 h before the event	ohsome API v2 (HeiGIT)	buildings, roads, bridges, places, health facilities

EMS maps are used **only** to score results (`waterguard/validate.py`, never imported by the pipeline). Each run audits its OSM snapshot: it fails if any feature was edited after the snapshot time (case study: 0 edited; latest edit in snapshot 26 Jul 2026). No training dataset is used (we chose the copilot as the AI component).

3. Method

3.1 Terrain. GLO-30 is hydrologically conditioned (least-cost breaching) with WhiteboxTools. We derive D8 flow directions, a stream network (≥ 2 km² upstream area), **HAND** (height above nearest drainage), slope and aspect.

3.2 Radar change (Sentinel-1). For every orbit track with an image after the event, the first post-event image is compared with up to five earlier images of the **same track** (same viewing geometry, 12-day repeat), as recommended for SAR change detection. After 5×5 multilooking (OpenCV), each pixel gets a z-score $|\text{post} - \text{mean(pre)}| / \text{std(pre)}$, with a 0.75 dB std floor. Pixels that normally fluctuate need a bigger change to count. Debris can brighten (boulders) or darken (mud, water), so the absolute value is used. Pixels in **layover or shadow** for that track, predicted from slope/aspect and the pass direction, are excluded. Tracks and polarisations are averaged. Open water is VV < −18 dB and VH < −25 dB.

3.3 Optical change (Sentinel-2). Monsoon cloud rarely leaves a clean scene, so we build per-pixel cloud-free median composites from the Scene Classification Layer over 45 days before and 40 days after the event. Reflectance is harmonised for processing baseline ≥ 04.00. Stripped vegetation shows as NDVI loss and new water or wet sediment as NDWI gain.

3.4 Fusion (OpenCV). A pixel is impacted if it is on the valley floor (HAND ≤ 80 m) **and** shows change: optical where both composites are clear, radar elsewhere (smoothed $|z| \geq 1.0$). Then come morphological opening/closing, a rule that patches must touch the river channel (a flood arrives from the river, not mid-slope) and a 2 ha minimum. Clear-sky ground that is bare or water after the event and connected to ≥ 5 ha of detected change is added. This captures the riverbed and gravel bars the surge ran through, which had no vegetation to lose. Ground that was bare *before* the event too, added only by this connectivity, is kept as a separate **lower-confidence riverbed** class.

3.5 Damage. A building, bridge or health facility is *likely affected* if it intersects impact and *possibly affected* within 20 m. We also report road length inside impact, by class.

3.6 Who is cut off. OSM roads become a graph (every vertex a node, so mid-way junctions count) for two networks: *vehicle* (motorable roads and tracks) and *foot* (plus paths, which matter in the hills). Impacted segments and likely-damaged bridges are removed. Destinations are towns/cities, health facilities and road exits at the area's edge. Settlements are OSM places plus clusters of ≥ 8 buildings without a named place, so unmapped hamlets count. Each settlement is tested through up to 25 of its own buildings, each snapped to its nearest road, so one flooded junction next to the place's map point cannot strand a whole suburb. Buildings are put in a canonical (coordinate) order before sampling, so an independent re-run samples the same houses: an independent verifier caught this when its fresh OSM download arrived in a different order and Syabru Bensi's estimate came out 988 instead of 1,200 people stranded. A settlement is **cut off** if none of its buildings can reach any destination after the flood, and **partly cut** if some can't or it lost a destination type. We report the share of its buildings with no road left and **people stranded** = people × that share. We also compute road distance to the nearest health facility before and after. People = buildings × 4.37 (Nepal 2021 census household size), always labelled as an estimate.

3.7 Flood path (bonus). From any clicked point, snapped to the nearest stream, we follow D8 steepest descent downstream and list settlements within 400 m of the path whose lowest part is ≤ 80 m above the river, ordered by distance. Times use an *assumed* 4 m/s surge and are labelled as such.

3.8 AI component: situation-report copilot. An LLM (Amazon Bedrock, default Amazon Nova Pro; or any OpenAI-compatible model) with six read-only tools over the run's results. Rules: answer in the user's language (English/Nepali) and copy every number from a tool result. A **verifier** extracts every number from the draft (ASCII or Devanagari digits) and checks it against the tool results of that conversation. On failure the model gets one

retry naming the bad numbers; numbers that still fail are masked `[unverified]` and flagged in the UI. Without an LLM, a deterministic router answers the common questions from the same tools. The one-page report is template-based, so its numbers are exactly the pipeline's.

4. Case study: EMSR927 comparison

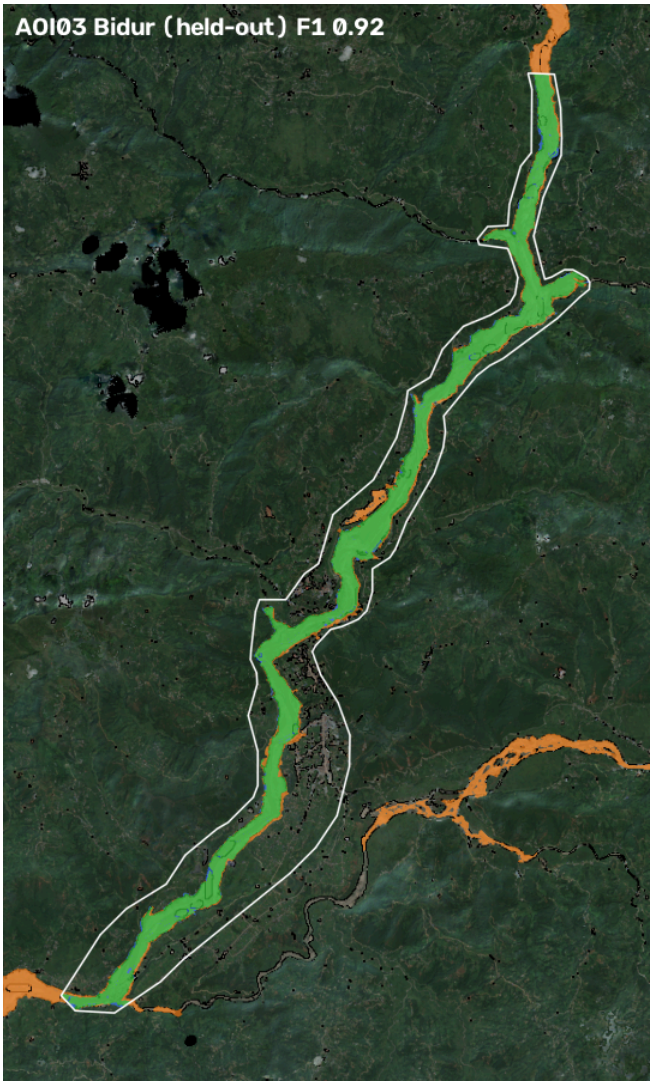
EMS delivered maps for four areas (AOI01 Syapru Besi, AOI02 Timure, AOI03 Bidur, AOI05 Phosretar; AOI04/06 were cancelled for cloud). EMS mapped the event as debris-flow/landslide footprints ("observed event"), plus graded building and bridge damage from very-high-resolution optical imagery. We score inside each AOI, excluding EMS "not analysed" areas.

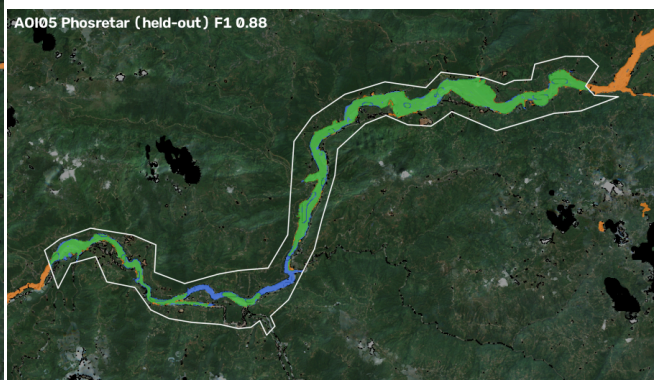
Honest protocol. Thresholds were tuned on **AOI01 + AOI02 only**. AOI03 and AOI05 are **held out**. The patch-size limits were set with false-alarm tests that use no EMS data: a *placebo date* (2 ha minimum patch) and an unaffected *control tributary*, Tadi Khola (5 ha minimum seed for riverbed growth; with 2 ha seeds, growth ran up Tadi's pre-existing gravel bed, 180 ha of false impact). As a baseline we mark the whole valley floor (HAND ≤ 80 m) as hit. This shows how much the satellite change evidence adds, since EMS AOIs are drawn tightly around the damaged valley.

EMS area	Split	EMS impact (ha)	Ours (ha)	Precision	Recall	F1	IoU	Valley-only baseline F1
AOI01 Syapru Besi	tuning	111.1	108.4	0.95	0.93	**0.94**	0.89	0.86
AOI02 Timure	tuning	129.4	109.6	0.98	0.83	**0.90**	0.81	0.89
AOI03 Bidur	held-out	701.4	775.7	0.88	0.97	**0.92**	0.85	0.56
AOI05 Phosretar	held-out	639.3	555.6	0.95	0.82	**0.88**	0.79	0.53

EMS area	EMS damaged/destroyed building points on our impact map	Destroyed	Damaged	Possibly damaged	EMS damaged bridges: in pre-event OSM / flagged by us
AOI01	404/433 (93%)	322/323	31/32	51/78	5 / 5 of 5
AOI02	394/431 (91%)	360/372	26/33	8/26	0 / 0 of 1
AOI03	2918/3058 (95%)	2473/2479	234/246	211/333	20 / 20 of 24
AOI05	567/763 (74%)	358/397	93/116	116/250	13 / 11 of 17

Do the results depend on the EMS calibration? Hardly. With the thresholds written *before* any EMS label was looked at (HAND ≤ 60 m, NDVI drop 0.10, radar |z| ≥ 1.1, 15 px smoothing) plus the riverbed rule, F1 on AOI01/02/03/05 is **0.89 / 0.80 / 0.92 / 0.86**, against 0.94 / 0.90 / 0.92 / 0.87 tuned (same cached features). The riverbed rule is the part that matters (without it: 0.83 / 0.53 / 0.85 / 0.54). It has a physical basis: a debris-flow channel is bare ground after the event, whether or not it was vegetated before. EMS maps never enter a run: they are read only by `validate.py`.





Held-out areas. Green = mapped by both, orange = only WaterGuard, blue = only EMS, white = EMS area outline. Orange outside the outline is outside what EMS analysed (e.g. the Tadi Khola, bottom right in Bidur: see Limitations).

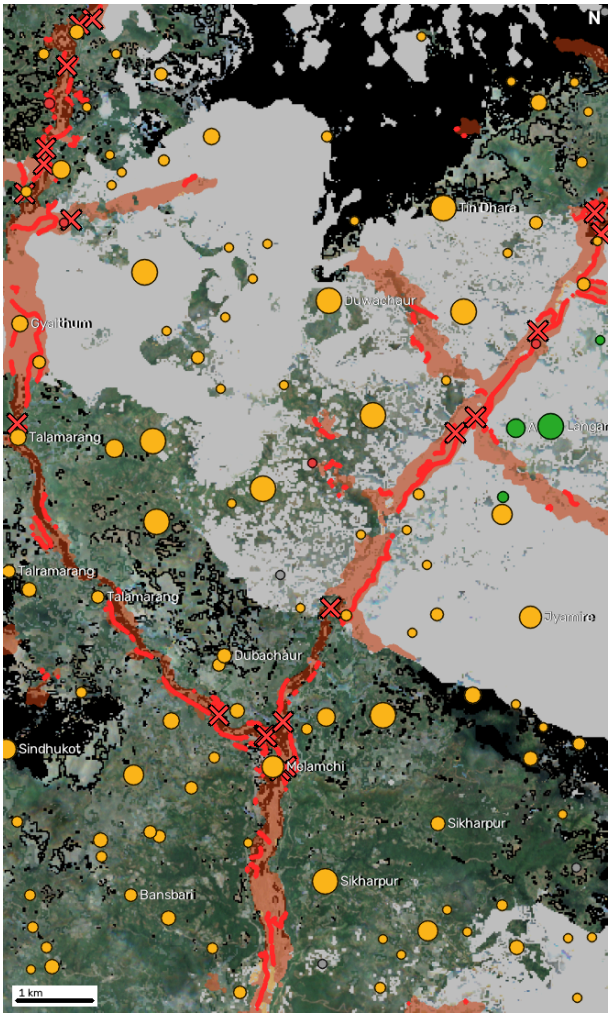
False-alarm tests (should map ~nothing):

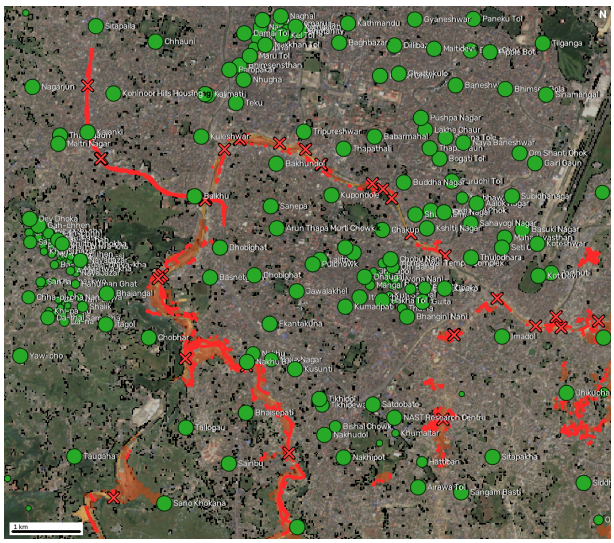
Test	Area (km²)	Impact mapped (ha)	of which riverbed, lower confidence (ha)	Buildings likely affected	Bridges likely damaged
placebo date (Syapru Besi, 20 Jul 2026)	29.8	0.0	0.0	0	0
control valley (Langtang Khola, real event date)	44.8	0.0	0.0	0	0
control tributary (Tadi Khola, real event date)	103.5	23.6	0.0	0	0

Copilot evaluation (`scripts/eval_copilot.py`, Amazon Nova Pro on Bedrock, Syapru Besi run, 13 questions: 9 English, 4 Nepali; transcript in `docs/copilot_eval.json`). **13 of 13 answers fully verified (106 numbers), no retries.** In an earlier run, one answer (a 5-line situation report) multiplied buildings by household size on its own. The verifier rejected the number, the retry repeated it and it was masked as `[unverified]`, so no invented figure reached the user. The pipeline now computes that figure itself (`people_est_in_likely_affected_buildings`). Asked "How many people died?", the copilot says the maps contain no casualty data and gives no number. Each answer takes 2–7 s.

Another event, untouched by tuning: Melamchi flood, 15 Jun 2021. We ran the same code, with no changes, on a different year, catchment and Sentinel-2 processing baseline (bbox 85.54–85.62 E, 27.80–27.92 N; 266 s cold). It mapped 954 ha of impact, 855 buildings likely affected and 20 bridges likely damaged. **Melamchi Bazar**, the town devastated in that flood, loses road access to its health post (12 km before, no route after), with 56% of its buildings left without a road. The first post-event radar image was 20 Jun 2021, five days after the flood. We have no EMS-style reference for this event, so this check is qualitative.

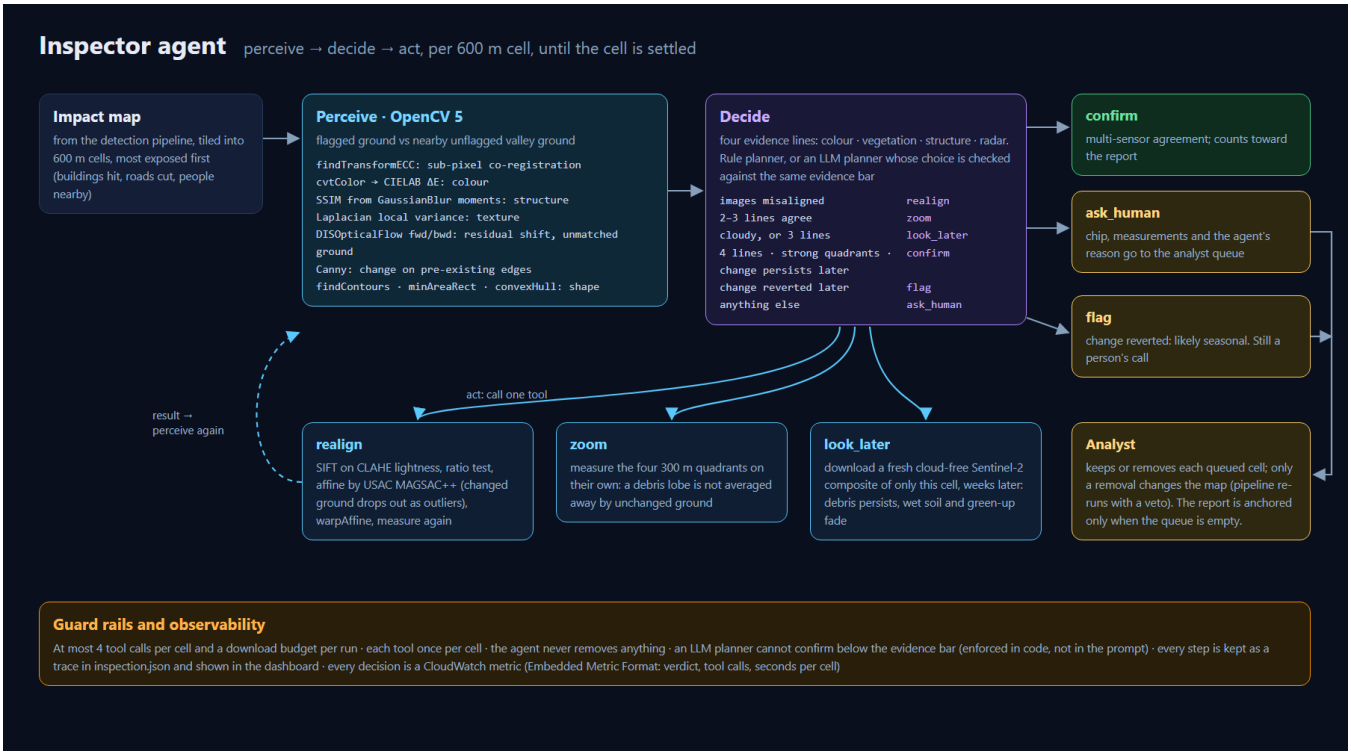
A flat, urban flood: Kathmandu Valley, 27–28 Sep 2024. This is the opposite of a Himalayan gorge, and it exposed two weaknesses that we fixed before freezing the code. First, roofs and streets are "bare" in Sentinel-2, so riverbed growth ran through the city (1,062 ha, 34,502 buildings "hit"). Pre-event OSM building footprints now block riverbed growth: 281 ha, along the Bagmati, Nakkhu, Balkhu and Manohara, the rivers that overflowed. Second, one flooded junction next to a place's map point stranded whole suburbs (Balkhu, Kalanki). The building-sampled cut-off test (3.6) fixed that: no settlement is cut off in the redundant city grid, and about 7,200 people's buildings lost road access, mostly near the Manohara. Both fixes cost a little on the EMS areas (Phosretar destroyed-building recall 96% → 90%). We kept them, because a judge's area can be anywhere.





5. Inspector agent (OpenCV 5)

A per-pixel map is a first draft. The inspector agent (`waterguard/inspector.py`, `waterguard/vision.py`) re-examines it in 600 m cells, most exposed first (buildings hit × 10 + road segments cut × 25 + people nearby × 0.05), in a perceive → decide → act loop that runs until each cell is settled or 4 tool calls are spent.



5.1 Perception. Every measure compares the flagged ground in the cell with nearby *unflagged* valley ground (HAND 3–80 m, at least 50 m from any flagged pixel), because a whole scene can brighten, dry out or green up between two dates; only the contrast counts.

Measure	OpenCV 5	What it catches
Co-registration	`findTransformECC` (translation, on Sobel gradient magnitude so the flood's brightness change doesn't drive it), `warpAffine`	sub-pixel offsets between the before and after composites
Colour change	`cvtColor` to CIELAB, ΔE per pixel	grey debris and sand replacing forest or fields
Structure	SSIM from `GaussianBlur` moments on L*	new surface pattern (boulders, scour)
Texture	`Laplacian`, local variance ratio	smoothed or roughened ground
Correspondence	`DISOpticalFlow` both ways, `remap`; forward–backward error and median flow over reference ground	ground that only shifted (consistent flow) vs ground that was replaced (no match); misalignment left after ECC
Edge artefacts	`Canny` + `dilate` on the before image	"change" that sits on edges that already existed (a misregistration signature)
Shape	`findContours`, `minAreaRect`, `convexHull`	elongated, solid debris tongues vs speckle
Analyst chip	`resize`, CLAHE, `applyColorMap` (TURBO), `drawContours`, `putText`, `hconcat`	before / after (aligned) / ΔE with the flagged outline

Together with Sentinel-2 NDVI loss and Sentinel-1 |z| these give four **evidence lines**: colour (ΔE contrast ≥ 12), vegetation (NDVI-loss contrast ≥ 0.10), structure (SSIM-dissimilarity contrast ≥ 0.15) and radar (|z| contrast ≥ 0.8), set on AOI01 + AOI02 and the Tadi Khola control.

5.2 Tools. *realign*: when ECC does not converge (or the dense flow over reference ground still shows > 1.5 px), SIFT keypoints on CLAHE-equalised lightness (upsampled ×2), Lowe's ratio test, and an affine model fitted with **USAC MAGSAC++** (`estimateAffine2D(method=USAC_MAGSAC)`). Keypoints on changed ground have no true match and fall out as outliers. Models with more than 2 % scale change, 1° rotation, shear or a 6 px shift are rejected (Sentinel-2 to Sentinel-2 is a pure shift). *zoom*: the same measures on each 300 m quadrant, so a debris lobe is not averaged away by unchanged ground; the cell is confirmed if quadrants where all four lines agree hold ≥ 60 % of its flagged ground. *look_later*: a new cloud-free Sentinel-2 composite of only this cell's window, from 40 days after the event, measured again: debris and scour persist; wet soil, turbid water and green-up do not.

5.3 Decisions and guard rails. Confirm when all four lines agree, when strong quadrants dominate, or when three lines agree (two under cloud) and the colour change persists weeks later. Flag (never remove) when the ground reverted. Everything else goes to an analyst with the chip, the measurements, the agent's reason and the full trace. With `--planner llm`, a language model chooses the next tool from the same state; the evidence bar, the one-use-per-tool rule and the step budget are enforced in code, so the model can steer the inspection but not lower the bar. The agent never removes impact: only an analyst's *Remove* does, by re-running the pipeline with that cell vetoed, and a report is anchored only when the queue is empty.

5.4 Evaluation. `python -m waterguard.validate --inspection` runs the agent on all four EMS areas and the Tadi Khola control. A cell is real damage when ≥ 50 % of its flagged pixels are inside EMS observed-event polygons.

	Cells	Confirmed	of which real	Real damage area auto-confirmed	Not-in-EMS area sent to a human	Later looks	realign / zoom calls
Tuning (AOI01, AOI02, Tadi Khola)	30	13	100 %	52 %	100 % (8 of 8 cells)	16	18 / 21
Held-out (AOI03, AOI05)	113	67	**98.5 %**	**68.6 %**	40 % (1 of 2 cells)	31	12 / 43

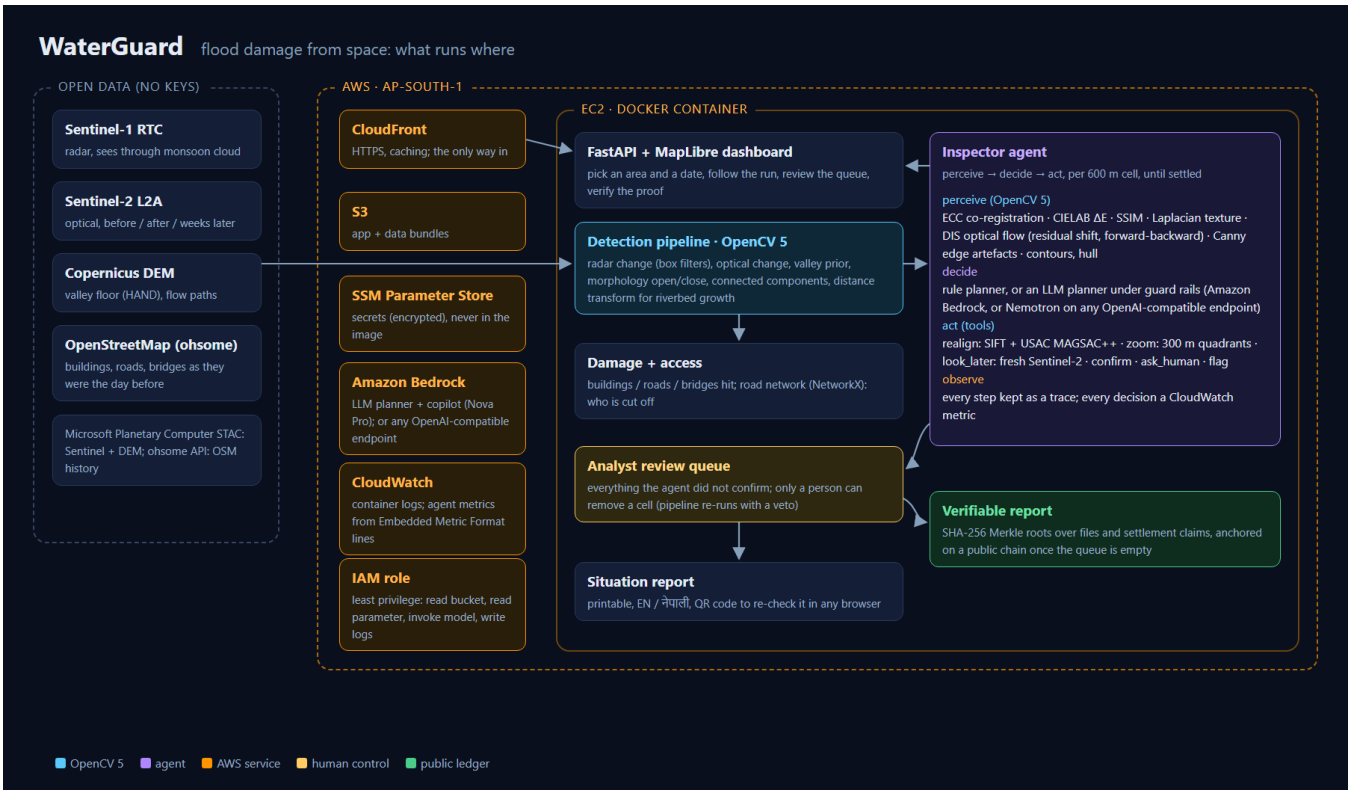
The agent before the realign and zoom tools (ECC only) scored 98.5 % / 66.7 % with 34 later looks on the same held-out cells. **Failure cases.** One held-out cell in Bidur (1.5 ha) that EMS did not map was confirmed: colour, vegetation and structure agreed (radar did not), zoom found no dominant quadrant, and the colour change persisted in the later image. The 8 Tadi Khola cells show vegetation, structure and sometimes radar change from the monsoon, but never the ΔE of fresh debris; colour is the line that separates them, which is why the agent will not confirm without it. In AOI05 (Phosretar) 30 of 51 scored cells still go to a person: most show only one or two lines (typically structure, or structure and radar) and the agent judges that too weak for a later image to settle.

5.5 Stress test: misaligned imagery. `python -m waterguard.validate --robustness` shifts the after bands of every scored cell by 0, 1.5, 3 and 5 px and settles each cell with and without the realign/zoom tools, with no later looks (so no downloads, and only perception differs). Held-out areas:

Shift	0 px	1.5 px	3 px	5 px
Real damage auto-confirmed, ECC only	49.0 %	31.3 %	4.2 %	0.4 %
Real damage auto-confirmed, with realign + zoom	52.9 %	55.6 %	50.6 %	50.9 %
Cells realigned by SIFT + MAGSAC++	11	37	84	97
False confirmations (either)	0	0	0	0

Misregistration does not produce false confirmations here, because each measure is a contrast with reference ground that moved too. It blinds the agent instead: at 3 px the ECC-only agent sends 108 of 113 cells to an analyst. Unit tests (`tests/test_vision.py`) check that realign recovers a 3.2 / 1.7 px shift with a third of the ground replaced (ECC fails on it), that it refuses unrelated images, and that the forward–backward flow separates shifted from replaced ground.

6. Architecture and AWS deployment



One Docker image (`Dockerfile`, pinned `requirements.txt`) holds the FastAPI dashboard, the pipeline and the agent. On AWS (`deploy/aws_deploy.sh`, `deploy/user-data.sh`) it runs on one EC2 `t3.xlarge` behind **CloudFront** (HTTPS; the security group admits only CloudFront's origin-facing prefix list; IMDSv2 required). The boot script pulls the app and data bundles from **S3** and the secrets from **SSM Parameter Store** (encrypted), so no keys are in the image or the repo. The LLM planner and copilot call **Amazon Bedrock** through the instance role. Container logs go to **CloudWatch Logs** (`/waterguard/app`); the inspector writes one Embedded Metric Format line per decision, which CloudWatch turns into the `WaterGuard/Inspector` metrics (cells by verdict and planner, tool calls, seconds per cell, later looks per run) without metrics API calls. The IAM role is limited to reading the deploy bucket and `/waterguard/*` parameters, invoking models and writing `/waterguard/*` logs. Only one flood run executes at a time; others queue. Reproducibility: every run's outputs are fingerprinted (SHA-256 Merkle roots, `waterguard/ledger.py`); independent re-runs on a Windows laptop and on an AWS Linux server reproduce the settlement-claims root byte for byte.

7. Responsible use

- **Decision support, not a decision.** Outputs are labelled satellite-derived and unverified on the ground; people counts are estimates and are labelled so.
- **Humans decide removals.** The agent can confirm or queue, never delete; every analyst decision is recorded with who and when, and published reports say how many cells were left unreviewed.
- **No invented numbers.** The copilot can only quote numbers present in tool results (number verifier); the one-page report is template-based.
- **Open data and open code only,** so anyone can re-run a report and check it; the EMS reference maps are used only for scoring.

- **Privacy.** Inputs are public satellite imagery at 10 m and public OpenStreetMap; no personal data is collected. The public demo's review queue is read-only.
- **Known failure modes are published** (sections 5.4 and 9), including where the agent defers.

8. Dashboard, report and copilot



The dashboard (`python -m waterguard.server`) takes a drawn box and a date and shows progress, then layers: impact, Sentinel-2 before/after, radar change, buildings hit, roads cut, bridges, and settlements coloured by road status. A side panel has the situation tiles, cut-off and bridge lists, the copilot, the one-page report (EN/NE), the flood-path tool and a *Basis* tab. That tab lists every satellite image used and the OSM audit.

9. Limitations: what WaterGuard cannot do

- **No early warning.** Sentinel-1 revisits each track every 12 days (two tracks here, so a new look every few days) and Sentinel-2 every 5 days, cloud permitting. WaterGuard maps damage *after* the fact. It cannot warn of a glacier collapse minutes ahead. In the case study the first post-event radar image came about 2.6 days after the flood (28 Aug 12:21 UTC).
- **Radar blind spots.** Steep slopes facing towards or away from the satellite (layover/shadow) cannot be assessed from that track. We mask them rather than guess. Narrow gorges are hard at 10 m.
- **Radar-decided impact is coarser.** Under cloud, the radar evidence is smoothed over ~200 m to beat speckle, so radar-decided patches come out as wider bands than optical ones (visible in the Melamchi run).
- **Clouds.** During the monsoon, optical change is available only where the composites found clear pixels. Elsewhere the coarser radar evidence decides. The Basis tab reports the split.
- **Building-level damage is inferred, not seen.** At 10 m, a building on mapped impact is *likely affected*. We cannot tell destroyed from damaged. EMS used 0.3–0.5 m imagery for that.
- **OSM completeness.** Unmapped roads, trails, bridges or buildings are invisible to the damage and cut-off analysis, and connectivity is only as good as the pre-event map. Areas outside OSM coverage give "no mapped road".
- **People are estimates** (buildings × 4.37 household size): it counts non-residential buildings, ignores seasonal migration and is not a census.
- **The area boundary matters.** "Cut off" means no route to a town, a health facility or a road leaving the drawn area. A small box can hide a detour that exists outside it.
- **Tributary riverbeds.** Bare gravel that connects to detected change is added to the impact map. At a confluence this can run up a tributary's riverbed that was already bare before the event. In the Bidur run the Tadi Khola bed is flagged this way, and from the allowed data we cannot tell whether a monsoon high flow hit it. Such pixels are a separate, lower-confidence class (yellow on the map) and are reported as such.
- **Cities.** Flooding between buildings is hard to see from radar (double bounce) and 10 m optical pixels mix roofs and water. In Kathmandu we map flooding along the river corridors, not street-level water.
- **Valley-floor prior.** Impact is only mapped below 80 m HAND. Landslides high on the slopes, which EMS also mapped, are out of scope.
- **Tuned on one event.** Thresholds come from two EMSR927 areas and were checked on two held-out areas plus false-alarm tests, but not on other flood types (e.g. plains riverine floods).
- **The copilot can still be wrong in words.** The verifier guarantees numbers, not the reasoning around them. The Nepali report templates have not yet been reviewed by a native speaker.

- **The inspector is tuned on one event too.** Its evidence lines were set on AOI01/02 and one control valley; held-out precision rests on 67 confirmed cells and only 2 held-out cells outside EMS, so its false-confirmation rate is not yet well measured.
- **Run time.** A cold run over about 30 km^2 takes 3–5 minutes, almost all of it downloading imagery from Planetary Computer.

10. Attribution and citations

Contains modified Copernicus Sentinel data 2026. Produced using Copernicus WorldDEM-30 © DLR e.V. 2010–2014 and © Airbus Defence and Space GmbH 2014–2018 provided under COPERNICUS by the European Union and ESA; all rights reserved. © OpenStreetMap contributors (ODbL), via the ohsome API (HeiGIT). Validation only: European Union, Copernicus Emergency Management Service data (EMSR927). Lindsay, J. B. (2016), *Whitebox GAT*, Computers & Geosciences 95. Nobre, A. D. et al. (2011), HAND, *J. Hydrology* 404. National Statistics Office Nepal (2021), *National Population and Housing Census 2021* (average household size 4.37).